

Relatório Técnico: Validador Formal em Três Níveis

Tema Escolhido: Tema 2 - Validador de Logs e Protocolos

Autores: Andrey Koch de Araujo, Pedro Lima Barriola, Julia Soupinski, Thiago

Github: <https://github.com/akda007/ValidadorFormal>

Video Relatório: <https://www.youtube.com/watch?v=gfvciKZmtV8>

1. Introdução e Contexto Aplicado

No escopo da engenharia de software e da arquitetura de sistemas distribuídos, a validação de fluxos de entrada representa uma operação fundamental de segurança e consistência operacional. Sempre que um sistema analisa um arquivo de log, um cabeçalho de protocolo de rede (como TCP, HTTP ou BGP) ou uma estrutura de transação financeira, ele está executando o reconhecimento de uma linguagem formal. O poder de expressividade e a eficiência computacional desse validador dependem estritamente do modelo teórico escolhido dentro da Hierarquia de Chomsky.

Este projeto prático consolida os conceitos fundamentais de linguagens formais e autômatos por meio do desenvolvimento e da análise comparativa de três reconhecedores de complexidades distintas. Adotando o cenário de um analisador de logs de infraestrutura, os três níveis operam de forma integrada: o primeiro gerencia o ciclo básico de autenticação, o segundo gerencia a integridade de blocos lógicos aninhados e o terceiro estende a validação para o controle rigoroso e balanceado de múltiplos eventos correlacionados.

2. Nível Regular: Linguagem Regular (LR)

O nível de Linguagem Regular é responsável por validar a conformidade sequencial básica do ciclo de vida de uma conexão em um servidor de aplicação. A especificação do protocolo determina que uma sessão válida deve seguir uma ordem estrita de operações elementares.

- **Descrição semântica:** A cadeia de log deve obrigatoriamente iniciar com o evento de LOGIN, ser imediatamente seguida por um evento de validação AUTH, admitir zero ou múltiplas requisições intermediárias do tipo REQUEST e ser formalmente encerrada pelo token LOGOUT.

- **Definição Formal Teórica:** $L = \{ \text{LOGIN} . \text{AUTH} . (\text{REQUEST})^n . \text{LOGOUT} \mid n \geq 0 \}$
- **Alfabeto da Linguagem (Σ):** $\Sigma = \{ \text{LOGIN}, \text{AUTH}, \text{REQUEST}, \text{LOGOUT} \}$

Para este nível, o modelo computacional adequado é um Autômato Finito Determinístico (DFA), que opera com memória estática representada unicamente por seus estados finitos, garantindo processamento em tempo linear em relação ao tamanho da entrada.

3. Nível Livre de Contexto: Linguagem Livre de Contexto (LLC)

Subindo na hierarquia de poder computacional, o segundo nível valida a integridade estrutural de transações complexas que podem ocorrer dentro do sistema de logs. Em sistemas de arquivos modernos ou bancos de dados relacionais, operações necessitam de isolamento em blocos delimitados.

- **Descrição semântica:** Análise de blocos aninhados de transação onde cada token de abertura BEGIN exige um token correspondente de fechamento END. O sistema permite o aninhamento arbitrário de transações internas, contanto que a ordem de fechamento preserve perfeitamente a simetria de escopo e não existam blocos órfãos.
- **Definição Formal Teórica:** $L = \{ \text{BEGIN}^n \text{END}^n \mid n \geq 1 \}$
- **Alfabeto da Linguagem (Σ):** $\Sigma = \{ \text{BEGIN}, \text{END} \}$

Como linguagens regulares não possuem capacidade de contagem ilimitada, este problema requer um Autômato com Pilha (PDA). A memória do tipo LIFO (Last-In, First-Out) permite armazenar os contextos de abertura abertos para posterior casamento estrutural e validação ao encontrar os delimitadores de encerramento.

4. Nível Recursivo: Linguagem Recursiva (R)

O nível mais elevado abordado no projeto lida com cenários onde múltiplas entidades dependentes precisam manter uma paridade exata e simultânea, ultrapassando os limites físicos de estruturas puras de pilha.

- **Descrição semântica:** Validação de um trio balanceado de eventos operacionais críticos. A cadeia de logs reflete um protocolo distribuído que exige que para cada sessão iniciada com um número 'n' de comandos OPEN, ocorra exatamente a mesma quantidade 'n' de confirmações COMMIT e, posteriormente, a mesma quantidade 'n' de encerramentos de conexão CLOSE.
- **Definição Formal Teórica:** $L = \{ \text{OPEN}^n \text{COMMIT}^n \text{CLOSE}^n \mid n \geq 1 \}$

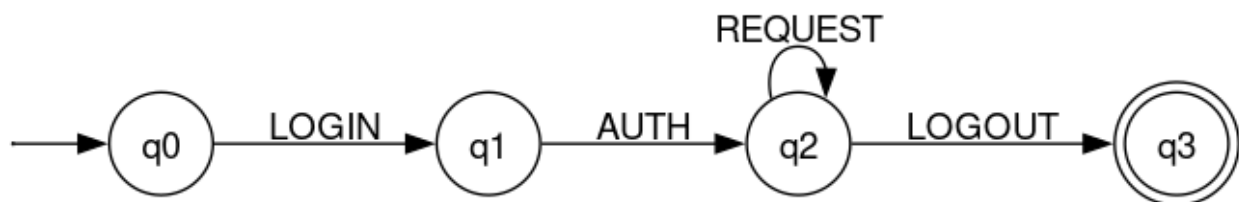
- **Alfabeto da Linguagem (Σ):** $\Sigma = \{ \text{OPEN}, \text{COMMIT}, \text{CLOSE} \}$

Esta linguagem descreve um problema clássico de não-livre-de-contexto (semelhante ao problema conceitual $a^n b^n c^n$). A validação necessita de uma Máquina de Turing (MT) que manipula uma fita bidirecional de leitura e escrita para realizar marcações temporárias e validações cruzadas nos três subgrupos da cadeia.

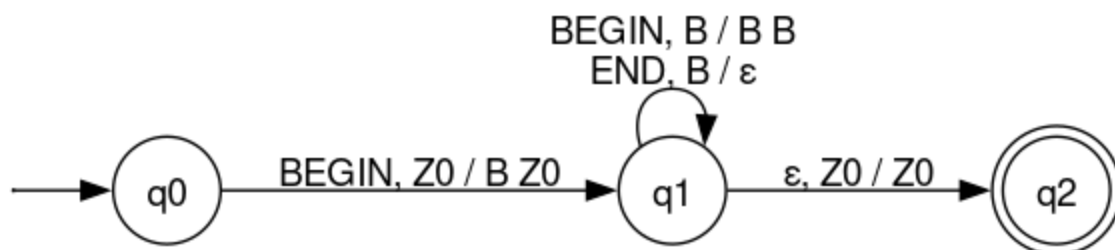
5. Modelos Formais e Tabelas de Transição

5.1. Diagramas do DFA, PDA e Turing

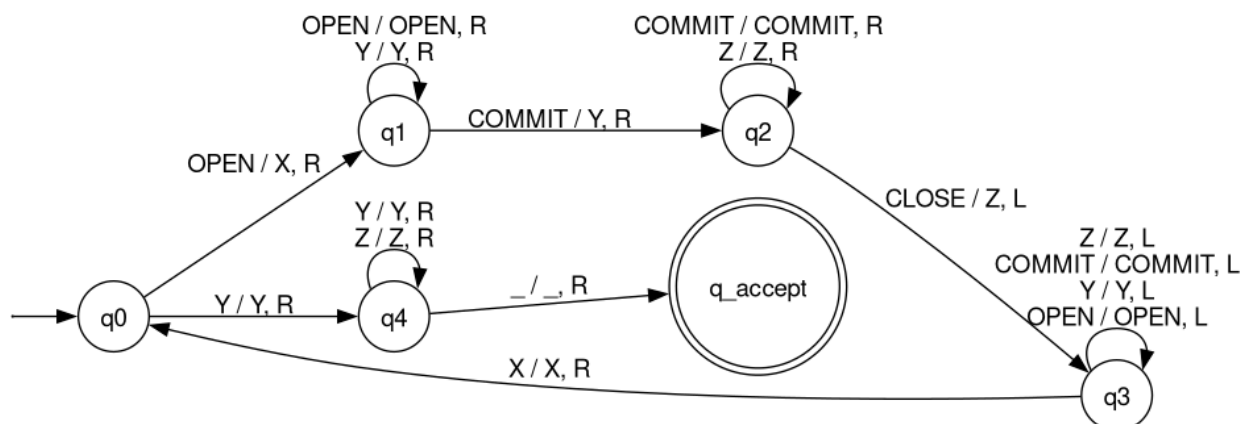
DFA Regular:



PDA Livre Contexto:



Turing Recursiva:



5.2. Tabela de Transição do DFA (Nível Regular)

Estado Atual	Símbolo Lido	Próximo Estado
q0 (Inicial)	LOGIN	q1
q1	AUTH	q2
q2	REQUEST	q2
q2	LOGOUT	q3 (Final)

5.3. Tabela de Transição do PDA (Nível Livre de Contexto)

Estado Atual	Símbolo Lido	Topo da Pilha	Próximo Estado	Ação na Pilha (Push)
q0 (Inicial)	BEGIN	Z0 (Fundo)	q1	[Z0, B]
q1	BEGIN	B	q1	[B, B]
q1	END	B	q1	[] (Desempilha)
q1	ϵ (Vazio)	Z0	q2 (Final)	[Z0]

5.4. Tabela de Transição da Máquina de Turing (Nível Recursivo)

Estado Atual	Símbolo Lido	Próximo Estado	Símbolo Escrito	Direção
q0	OPEN	q1	X	R
q1	OPEN / Y	q1	OPEN / Y	R
q1	COMMIT	q2	Y	R

Estado Atual	Símbolo Lido	Próximo Estado	Símbolo Escrito	Direção
q2	COMMIT / Z	q2	COMMIT / Z	R
q2	CLOSE	q3	Z	L
q3	Z/COMMIT/Y/OPEN	q3	Mantém símbolo	L
q3	X	q0	X	R
q0	Y	q4	Y	R
q4	Y / Z	q4	Y / Z	R
q4	_ (Branco)	q_accept	_	R

6. Decisões de Implementação

A implementação do sistema de validação seguiu de maneira estrita as diretrizes teóricas, priorizando construções limpas, legíveis e modulares orientadas a objetos em Python. As decisões fundamentais de engenharia incluem:

1. **Mapeamento Declarativo Baseado em Dados:** Foi completamente abolido o uso de condicionais aninhadas de fluxo (estruturas de if/else sequenciais) para representar as transições. O motor interno de cada autômato executa consultas dinâmicas em dicionários do Python. O comportamento matemático do autômato é guiado diretamente pelas estruturas de dados, agindo como um simulador purista.
2. **Rigor na Métrica de Passos:** Conforme explicitado no escopo do projeto, o contador de passos não mede iterações de laços do código hospedeiro ou linhas computadas. O incremento ocorre única e exclusivamente no momento em que a função matemática de transição (δ) é efetivamente acionada pela leitura ou movimentação do cabeçote.
3. **Processamento Simbólico:** As entradas textuais de log são pré-processadas e tokenizadas por delimitadores de espaços, transformando cadeias de palavras em símbolos puros de entrada equivalentes aos caracteres de fitas tradicionais.

7. Execução Passo a Passo e Análise de Casos

Abaixo, detalha-se o comportamento operacional interno dos motores com o rastreamento completo de cadeias válidas e inválidas para cada reconhecedor desenvolvido:

DFA (Cadeia Aceita: LOGIN AUTH LOGOUT)

```
Iniciando validação. Estado inicial: q0
Passo 1: Leu 'LOGIN' -> Mudou de q0 para q1
Passo 2: Leu 'AUTH' -> Mudou de q1 para q2
Passo 3: Leu 'LOGOUT' -> Mudou de q2 para q3
Cadeia ACEITA em 3 passos. Parou no estado final q3.
```

PDA (Cadeia Rejeitada: BEGIN END END)

```
Iniciando validação. Estado inicial: q0, Stack: ['Z0']
Passo 1: Read 'BEGIN', Pop 'Z0' -> State q1, Stack ['Z0', 'B']
Passo 2: Read 'END', Pop 'B' -> State q1, Stack ['Z0']
Rejeitado no passo 3: Transição inválida para a tupla ('q1', 'END', 'Z0').
```

8. Resultados Consolidados da Bateria de Testes

Cadeia de Teste Seleccionada	Classe	Resultado Esperado	Resultado Obtido	Passos Computados
LOGIN AUTH LOGOUT	LR (Regular)	ACCEPT	ACCEPT	3 passos
LOGIN AUTH REQUEST REQUEST LOGOUT	LR (Regular)	ACCEPT	ACCEPT	5 passos
LOGIN LOGOUT AUTH	LR (Regular)	REJECT	REJECT	2 passos

Cadeia de Teste Seleccionada	Classe	Resultado Esperado	Resultado Obtido	Passos Computados
BEGIN BEGIN END END	LLC (Livre de Contexto)	ACCEPT	ACCEPT	5 passos
BEGIN END END	LLC (Livre de Contexto)	REJECT	REJECT	3 passos
OPEN OPEN COMMIT COMMIT CLOSE CLOSE	R (Recursiva)	ACCEPT	ACCEPT	21 passos
OPEN COMMIT CLOSE CLOSE	R (Recursiva)	REJECT	REJECT	6 passos

9. Análise Crítica e Comparação Hierárquica

A execução coordenada da bateria de testes comprova de maneira empírica as fundações matemáticas contidas na tese clássica da Hierarquia de Chomsky ($LR \subsetneq LLC \subsetneq R$). A variação gritante no volume de passos executados expõe diretamente o preço computacional cobrado pelo aumento do poder de expressividade de um modelo.

O modelo **LR (DFA)** apresenta uma execução incrivelmente previsível e performática: ele varre os tokens estritamente uma única vez, em complexidade de tempo de pior caso linear, $O(N)$. No entanto, o DFA é completamente incapaz de lidar com estruturas com dependências de longa distância ou contagens simétricas, pois sua memória imediata satura-se de forma fixa nos estados pré-alocados.

O modelo **LLC (PDA)** supera esta barreira de contagem acoplando um mecanismo de memória elástica (Pilha). Ele consegue realizar o rastreamento perfeito de blocos indefinidamente profundos, mantendo um custo controlado de passos. Contudo, seu ponto fraco reside na própria propriedade LIFO: uma vez que os itens são empilhados, o acesso aos elementos inferiores exige o desempilhamento completo do topo. Devido a este isolamento, torna-se impossível parear e validar uma terceira estrutura dependente em paralelo.

O modelo **R (Máquina de Turing)** quebra as barreiras de isolamento através de uma fita irrestrita de leitura/escrita e controle livre de movimentação bidirecional. Esse poder irrestrito

permitiu validar com sucesso a linguagem do trio balanceado, algo impossível nos níveis anteriores. A penalidade, entretanto, reside na explosão do número de passos necessários: o vaivém do cabeçote para marcar e apagar símbolos gera um comportamento assintótico polinomial de alto custo para strings grandes, evidenciando por que a indústria adota modelos mais restritos sempre que o escopo permite.

10. Conclusão e Funcionalidades de Bônus

O projeto valida com sucesso os conceitos abstratos de Teoria da Computação aplicando-os a um cenário moderno de análise de engenharia de software. Como diferencial prático de engenharia de software e para consolidação dos pontos de bonificação opcionais previstos, foram desenvolvidas duas extensões acopladas:

- **Interface Unificada em Streamlit (Dashboard):** Centralizou os três validadores em um painel web dinâmico, fornecendo caixas de rastreamento retráteis onde o usuário visualiza instantaneamente os passos, as strings e o resultado lógico.
- **Análise Comparativa com o Módulo Nativo RE:** Foi desenvolvido um script específico para contrastar o DFA purista manual contra o mecanismo otimizado compilado em C da biblioteca padrão do Python. A análise demonstra o ganho comercial das engines de mercado, mas ressalta o valor do modelo manual acadêmico no fornecimento de transparência absoluta sobre as transições de estados e mensuração teórica de passos ocultos.